

FORMAT FOR PREPARATION OF PROJECT REPORT

FOR

Foundations of Artificial Intelligence

AI-BASED MAZE SOLVER USING

BREADTH-FIRST SEARCH (BFS)

AND

DEPTH-FIRST SEARCH (DFS)

A PROJECT REPORT

Submitted by

240701224 – KAMALESH V

240701232 – KARTHICK R



BONAFIDE CERTIFICATE

Certified that this project report “**AI-Based Maze Solver Using Breadth-First Search (BFS) and Depth-First Search (DFS)**” is the bonafide work of “**Kamalesh V (240701224)**” and “**Karthick R (240701232)**” of the Department of Computer Science and Engineering, Rajalakshmi Engineering College, who carried out the project work for the subject **Foundations of Artificial Intelligence** during the academic year 2025-2026 under my supervision.

Guide: Muneeshwari P

SIGNATURE OF THE FACULTY INCHARGE

Muneeshwari P

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

ABSTRACT

Artificial Intelligence (AI) is concerned with building systems that perceive, reason and act to achieve goals, and search is one of its most fundamental problem-solving techniques. This project applies AI search to the maze solving problem, in which an agent must find a route from a start cell to a goal cell through a grid containing walls. The maze is modelled as a state space in which each open cell is a state and each legal move is an action. Two uninformed search algorithms are implemented. Breadth-First Search (BFS) uses a queue to explore the maze level by level and guarantees the shortest path, whereas Depth-First Search (DFS) uses a stack to follow one branch deeply before backtracking, needs less memory and does not guarantee the shortest path. The system is written in Python with a Tkinter interface that generates perfect and random mazes, animates the search with colour-coded cells and displays the final path recovered by parent tracking. A comparison module records path length, nodes visited and execution time. Tests on seven mazes show that BFS always returned a shortest path, for example 42 moves against 88 for DFS on a 20×20 maze, while DFS visited far fewer nodes in open mazes. The project is expected to give a clear, visual understanding of uninformed search and a base for heuristic algorithms such as A*.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
1	INTRODUCTION	1
2	LITERATURE REVIEW	5
3	PROPOSED SYSTEM	9
	3.1 ARCHITECTURE DIAGRAM.....	12
4	MODULES DESCRIPTION.....	14
	4.1 Maze Creation Module	14
	4.2 BFS Search Module.....	14
	4.3 DFS Search Module.....	15
	4.4 Path Reconstruction Module	16
	4.5 Visualization Module	16
	4.6 Comparison Module	17
5	IMPLEMENTATION AND RESULTS	19
	5.1 EXPERIMENTAL SETUP	19
	5.2 RESULTS	21
6	CONCLUSION AND FUTURE WORK.....	26
7	REFERENCES	28
	APPENDIX.....	30

CHAPTER 1

INTRODUCTION

1.1 Introduction to Artificial Intelligence

Artificial Intelligence (AI) is the branch of computer science that studies how to build machines and software capable of behaviour that would be called intelligent if a human performed it. Modern definitions describe AI as the design of agents that perceive their environment and take actions that maximise their chance of achieving a goal (Russell and Norvig, 2021).

Among these areas, **problem solving by search** is one of the oldest and most fundamental. Many practical problems, such as planning a route, solving a puzzle, scheduling tasks or playing a game, can be described as a search for a sequence of actions that leads from an initial situation to a desired one.

1.2 Intelligent Agents

An **agent** is anything that perceives its environment through sensors and acts upon it through actuators. A **rational agent** selects the action that is expected to maximise its performance measure, given what it has perceived so far. The task environment of an agent is commonly described using the PEAS framework: Performance measure, Environment, Actuators and Sensors.

For the maze solver developed in this project, the agent is the search program. Its performance measure is finding a valid path to the goal, preferably a short one, while examining as few cells as possible. The environment is a two-dimensional grid of open cells and walls that is fully observable, deterministic, static, discrete and known in advance. The actuators are the moves Up, Right, Down and Left, and the sensors are the map data that tell the agent which neighbouring cells are open. As the environment is fully known, the agent can

plan the whole route in advance, so it is a **problem-solving (goal-based) agent**.

1.3 Search Problems in AI

A search problem is defined formally by five components. The **initial state** is where the agent begins. The **actions** available in each state describe what the agent can do. The **transition model** states which state results from performing an action. The **goal test** checks whether a given state is the goal. The **path cost** assigns a numeric cost to a sequence of actions. A **solution** is a sequence of actions that leads from the initial state to a goal state, and an **optimal solution** is the one with the lowest path cost.

In the maze problem the initial state is the start cell, the goal state is the exit cell, each legal move between adjacent open cells is an action, and every move has a step cost of one. The path cost is therefore simply the number of steps taken.

1.4 State Space Search

The set of all states reachable from the initial state through any sequence of actions is called the **state space**. It can be viewed as a graph in which the nodes are states and the edges are actions. Search algorithms explore this graph by building a **search tree**: the root is the initial state, and expanding a node generates its successors. The efficiency of a search is described by the branching factor b (the number of successors per node), the depth d of the shallowest goal and the maximum depth m of the space.

Because the same state can be reached by several routes, a search over a graph must remember the states it has already reached, often called the explored set or visited set. Without it, the algorithm may loop forever. In a grid maze with R rows and C columns there are at most $R \times C$ states, so a search that records visited cells runs in time proportional to the number of cells.

1.5 Uninformed Search Algorithms

Uninformed (blind) search algorithms use only the information in the problem definition. They have no knowledge about which non-goal state looks more promising, and they differ only in the order in which they expand nodes. The commonly studied uninformed strategies are Breadth-First Search, Depth-First Search, Uniform-Cost Search, Depth-Limited Search and Iterative Deepening Depth-First Search. Table 1.1 summarises their main properties.

Table 1.1: Properties of uninformed search algorithms

Algorithm	Frontier	Complete	Optimal	Time	Space
BFS	FIFO queue	Yes	Yes (unit cost)	$O(b^d)$	$O(b^d)$
DFS	LIFO stack	Yes on finite graphs with visited set	No	$O(b^m)$	$O(bm)$
Uniform-Cost	Priority queue	Yes	Yes	$O(b^{(1+C^*/\epsilon)})$	$O(b^{(1+C^*/\epsilon)})$
Depth-Limited	LIFO stack	No	No	$O(b^l)$	$O(bl)$
Iterative Deepening	LIFO stack	Yes	Yes (unit cost)	$O(b^d)$	$O(bd)$

This project focuses on BFS and DFS, the two basic building blocks of graph search, which show the trade-off between optimality and memory very clearly.

1.6 Maze Solving Concepts

A maze is a network of passages separated by walls, and solving it means finding a route from an entrance to an exit. On a computer, a maze is most simply stored as a two-dimensional grid in which each cell is either open (free) or a wall (blocked). Movement is normally allowed in the four cardinal directions, so each open cell has at most four neighbours. A **perfect maze** contains no loops and no unreachable areas, so exactly one path exists between any two cells, whereas a maze with random walls can contain loops and

several alternative routes and may even have no solution at all (Buck, 2015). The same grid model is used in robot navigation and video-game path-finding.

1.7 Importance of BFS and DFS

Breadth-First Search expands the shallowest unexpanded node first, using a first-in first-out queue. It explores the maze in expanding layers around the start cell and therefore reaches the goal by a path with the smallest number of steps whenever every step costs the same. Its drawback is memory, since the frontier may hold many cells at once.

Depth-First Search always expands the deepest node in the frontier, using a last-in first-out stack or recursion. It follows one corridor as far as possible, and when it reaches a dead end it backtracks to the most recent junction that still has an unexplored branch. Its memory requirement is small, and it can reach a goal quickly in some layouts, but the path it finds may be much longer than the shortest one.

Both algorithms run in $O(V + E)$ time and form the basis of more advanced techniques: BFS leads to Dijkstra's algorithm and A* search, while DFS underlies topological sorting, cycle detection and iterative deepening (Cormen et al., 2009).

1.8 Project Motivation

Search algorithms are usually taught through pseudocode and complexity formulas, which makes it hard for students to see how differently BFS and DFS behave. A maze is an ideal visual laboratory: the spreading wave of BFS and the deep, winding exploration of DFS can be watched cell by cell. The motivation for this project is therefore to build a working tool that implements both algorithms in Python, animates them on a graphical interface and measures them objectively.

The project also has practical motivation, since grid-based path-finding is used in robotics, warehouse automation, game development and circuit routing. By comparing path length, nodes visited and execution time, the project turns abstract theory into measurable evidence.

CHAPTER 2

LITERATURE REVIEW

2.1 Artificial Intelligence Search Techniques

Search has been central to AI since its earliest years. Textbooks such as Russell and Norvig (2021), Nilsson (1998) and Poole and Mackworth (2017) classify search methods into uninformed strategies, which use no domain knowledge, and informed or heuristic strategies, which use an estimate of the remaining cost to guide the search. Pearl (1984) gave a detailed treatment of heuristic search and explained how the quality of a heuristic affects the number of nodes expanded. The best-known informed method is the A* algorithm of Hart, Nilsson and Raphael (1968), which combines the cost already spent with a heuristic estimate and is optimal when the heuristic is admissible.

Korf (1985) introduced iterative-deepening depth-first search, which repeats depth-limited DFS with an increasing limit. It obtains the optimality of BFS with the linear memory of DFS and shows how the two basic strategies can be combined. These studies establish BFS and DFS as the reference points against which other search techniques are measured.

2.2 Graph Traversal Algorithms

Graph traversal means visiting the vertices of a graph in a systematic order. The two fundamental traversals are BFS and DFS, and their theory is presented in detail by Cormen et al. (2009) and Skiena (2008). Both mark vertices as visited to avoid repetition and both run in $O(V + E)$ time when the graph is stored as adjacency lists. Tarjan (1972) demonstrated that DFS is a powerful tool for solving many graph problems in linear time, including finding biconnected components and strongly connected components, and Hopcroft and Tarjan (1973) extended the use of DFS to other graph

manipulation problems. Dijkstra (1959) generalised the idea of shortest-path search to graphs with non-negative edge weights by replacing the plain queue with a priority queue.

2.3 Applications of BFS

BFS is used wherever the shortest route in terms of number of steps is needed. Moore (1959) described a breadth-first procedure to find the shortest path through a maze, and Lee (1961) applied the same wave-propagation idea to route connections between components on circuit boards, an approach that is still known as the Lee algorithm. In network flow, Edmonds and Karp (1972) used BFS to find shortest augmenting paths, which gave the first polynomial bound for the Ford-Fulkerson method. BFS is also used for finding the fewest hops in computer networks, for crawling web pages level by level, for computing degrees of separation in social networks, and for garbage collection algorithms that copy reachable objects.

2.4 Applications of DFS

DFS is preferred when the whole structure must be explored or when memory is limited. Tarjan (1972) used it for connectivity analysis, and it is also used for topological sorting of tasks with dependencies, detecting cycles in directed graphs, solving puzzles by backtracking, generating mazes and checking whether one node is reachable from another. Many maze generators use a randomised depth-first carving process, which produces long winding corridors with few branches and is the method also used in this project to build perfect mazes (Buck, 2015). In constraint satisfaction problems such as Sudoku and the eight-queens puzzle, backtracking search is essentially DFS over the space of partial assignments (Russell and Norvig, 2021).

2.5 Maze Solving Systems

Maze solving has been studied both as a puzzle and as a benchmark for algorithms. Moore (1959) treated it as a shortest-path problem, and later work compared wall-following, random mouse, dead-end filling and recursive backtracking methods with graph search. Wall-following needs almost no memory but works only when the maze is simply connected. Dead-end filling removes dead ends until only the solution remains. Recursive backtracking is DFS in disguise and finds a path but not necessarily the shortest. BFS and its weighted variants are the standard methods when the shortest path is required. Educational tools and micromouse competitions rely on these ideas, with micromouse robots typically mapping the maze and then applying a flood-fill algorithm, which is BFS from the goal.

2.6 Robot Navigation Systems

Mobile robots often represent their surroundings as an occupancy grid in which each cell is marked free or occupied, which is exactly the maze model used here. Choset et al. (2005) describe how grid-based planners, roadmaps and potential fields are used to plan collision-free motion. On a grid, wavefront planners propagate a BFS wave from the goal and let the robot follow the decreasing values to the goal. For large or continuous environments, informed planners such as A* and its variants are used because they reduce the number of cells examined, but BFS and DFS remain the conceptual foundation and are still used for simple, small-scale or unweighted navigation tasks.

2.7 Game Path-Finding Systems

In video games, non-player characters must move across maps that are typically stored as grids or navigation meshes. Sturtevant (2012) introduced a set of benchmark grid maps taken from commercial games and used them to

compare path-finding algorithms in a standard way. Games generally choose A* because it finds short paths quickly, but BFS is still used for tasks such as computing movement range in tactical games, flood filling regions and spreading effects, and DFS is used for procedural level generation and for checking that a generated level is connected. These studies show that even the simplest uninformed algorithms have real practical value.

2.8 Comparison of Existing Methods

Table 2.1 compares the main search methods discussed in the literature. It shows that no single method is best for every situation: the choice depends on whether optimality, memory, speed or availability of a heuristic is the most important requirement.

Table 2.1: Comparison of existing path-finding methods

Method	Type	Shortest path	Memory	Typical application
BFS	Uninformed	Yes (unit cost)	High	Shortest path in unweighted grids, wavefront planning, web crawling
DFS	Uninformed	No	Low	Maze generation, backtracking, cycle detection, connectivity
Iterative deepening	Uninformed	Yes (unit cost)	Low	Large trees with unknown solution depth
Dijkstra	Uninformed, weighted	Yes	High	Road networks, routing protocols
A*	Informed	Yes (admissible heuristic)	High	Game path-finding, robot planning
Wall following	Local rule	No	Very low	Simply connected mazes, simple robots

The review shows that BFS and DFS are the base algorithms from which the more advanced methods are derived. A clear, measured comparison of the two on the same mazes, presented visually, is therefore a useful first step, and it is the gap that this project addresses.

CHAPTER 3

PROPOSED SYSTEM

Problem Statement

Given a rectangular maze represented as a grid of open cells and walls, together with a start cell and a goal cell, the problem is to find a path from the start to the goal that moves only through open cells and only in the four cardinal directions. Because different search strategies find different paths and examine different numbers of cells, the system must also compare the strategies objectively. The difficulty for learners is that the internal behaviour of BFS and DFS cannot be seen in ordinary program output. A system is therefore needed that solves the maze with both algorithms, shows the search visually and reports measurable results.

Objectives

- To model the maze as a state space search problem on a two-dimensional grid.
- To implement Breadth-First Search using a queue and Depth-First Search using a stack in Python.
- To reconstruct and display the final path from the start cell to the goal cell using parent tracking.
- To develop a Tkinter graphical interface with colour coding and step-by-step search animation.
- To compare BFS and DFS on path length, number of nodes visited and execution time.
- To support both perfect mazes and mazes with random walls, and to handle mazes that have no solution.

Working Principle

The system treats every open cell of the grid as a state and every legal move to a neighbouring open cell as an action with a cost of one. The search starts at the start cell and keeps a **frontier** of cells that have been discovered but not yet expanded. In BFS the frontier is a queue, so the cell that was discovered first is expanded first and the search spreads outwards in layers of equal distance from the start. In DFS the frontier is a stack, so the most recently discovered cell is expanded first and the search dives along one corridor until it reaches a dead end, after which it backtracks.

Each time a cell is discovered the algorithm records which cell it came from in a parent table. When the goal is removed from the frontier the search stops, and the parent table is followed backwards from the goal to the start to obtain the path. If the frontier becomes empty before the goal is reached, the maze has no solution.

Advantages

- **Visual learning:** colour-coded animation makes the difference between BFS and DFS immediately clear.
- **Objective comparison:** both algorithms run on exactly the same maze and are measured with the same metrics.
- **Guaranteed correctness:** BFS always returns a shortest path in an unweighted grid, and both algorithms report failure correctly when no path exists.
- **Flexibility:** mazes can be generated automatically or edited by clicking cells, so many scenarios can be tested.
- **Simplicity and portability:** the system uses only the Python standard library and runs on any operating system, including Windows.

- **Extensibility:** the modular design allows heuristic algorithms such as A* to be added with little change.

System Workflow

The system works as a pipeline of seven stages, each of which passes its output to the next: User Input, Maze Creation, BFS/DFS Engine, Path Search, Path Reconstruction, Visualization and Results. The stages are explained below.

Stage 1: User Input: The user chooses the maze type (perfect or random), the wall density for random mazes, and the algorithm to run. The user can also click on cells to add or remove walls, and can start the comparison of both algorithms with a single button.

Stage 2: Maze Creation: The maze creation module builds the grid, places the walls and fixes the start and goal cells. For a random maze each cell becomes a wall with the chosen probability. For a perfect maze a randomised depth-first carving process removes walls until every cell is connected without loops.

Stage 3: BFS / DFS Engine: The selected algorithm is called with the grid, the start cell and the goal cell. This stage decides which data structure manages the frontier: a queue for BFS or a stack for DFS. Because both algorithms share the same interface, they can be exchanged without changing the rest of the system.

Stage 4: Path Search: The engine repeatedly removes a cell from the frontier, marks it as visited, tests whether it is the goal, and otherwise adds its unvisited open neighbours to the frontier while recording their parent. The order in which the cells are expanded is stored so that it can be animated later.

Stage 5: Path Reconstruction: When the goal is reached, the parent table is followed from the goal back to the start and the resulting list is reversed to

give the route from start to goal. If the goal is not in the parent table, an empty path is returned to signal that no route exists.

Stage 6: Visualization: The Tkinter canvas draws the maze and replays the stored expansion order cell by cell in a light colour, and finally highlights the reconstructed path in a contrasting colour.

Stage 7: Results: The system reports the path length, the number of nodes visited and the average execution time, and in comparison mode displays the results of BFS and DFS side by side.

3.1 ARCHITECTURE DIAGRAM

Figure 3.1 shows the architecture of the proposed maze solver as a layered pipeline. Data flow from the top of the diagram to the bottom.

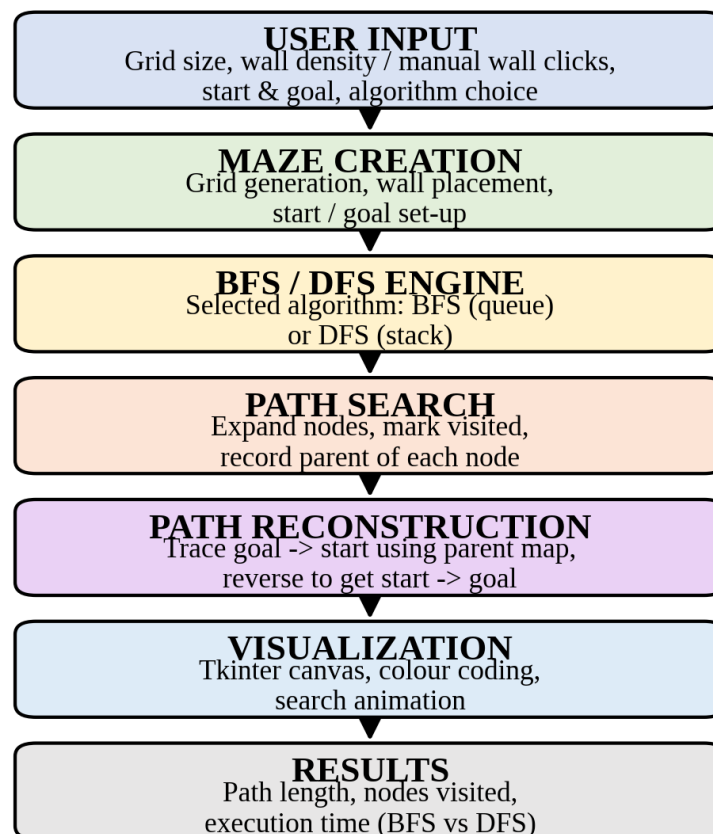
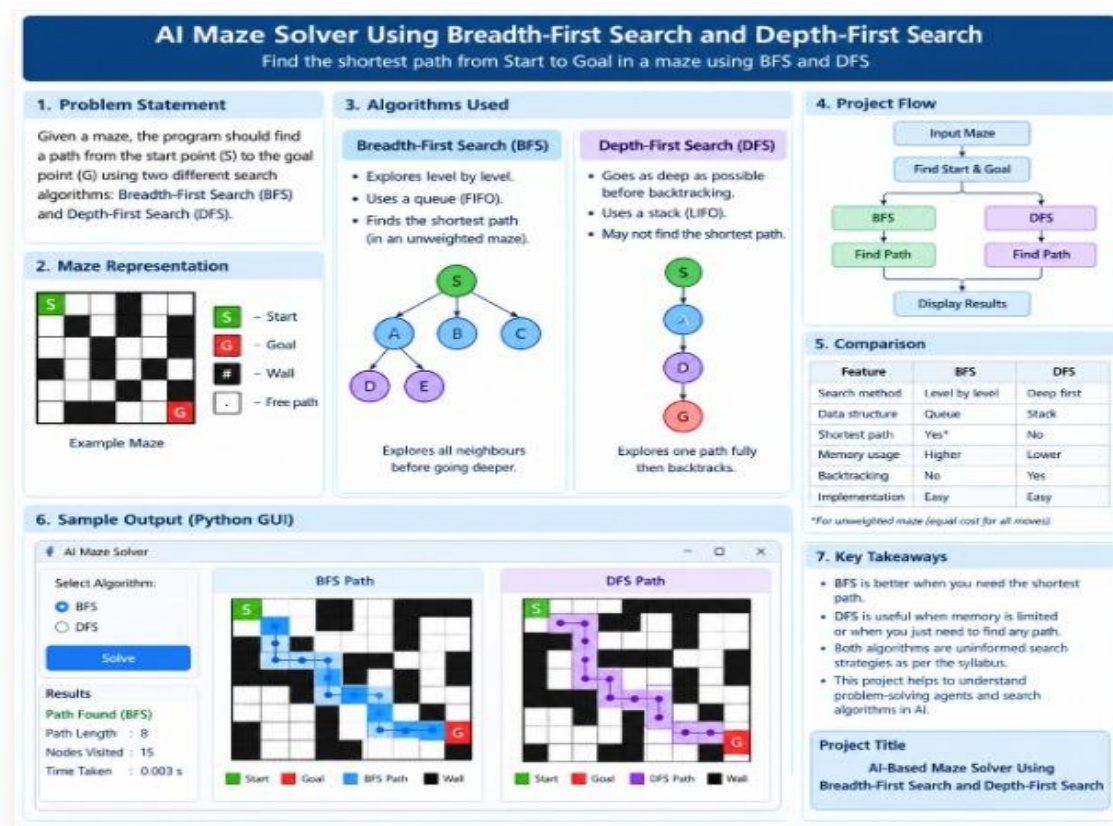


Figure 3.1: Architecture diagram of the AI-based maze solver

The **User Input** block collects the choices made through the graphical interface. The **Maze Creation** block converts those choices into a grid data structure and is independent of any search algorithm. The **BFS / DFS Engine** block is the decision point of the system: it takes the grid and runs the chosen algorithm. The **Path Search** block is the working part of the engine, where nodes are expanded and parents are recorded. Its output, the explored order and the parent table, feeds the **Path Reconstruction** block, which produces the final route. The **Visualization** block consumes both the explored order and the final route to draw the animation. Finally, the **Results** block collects the metrics and presents them to the user. Since each block communicates with the next only through simple data structures (a grid, a list of cells and a dictionary), any block can be replaced or improved without affecting the others.



CHAPTER 4

MODULES DESCRIPTION

The system is divided into six modules. Each module has a single responsibility and is implemented as one or more Python functions. This chapter describes the purpose, inputs, outputs and working of each module.

4.1 Maze Creation Module

Purpose: To build the grid on which the search is performed, including the placement of walls and the setting of the start and goal cells.

Inputs: Number of rows and columns, wall density (for random mazes), optional random seed, and manual wall edits made by the user.

Outputs: A two-dimensional list in which 0 represents an open cell and 1 represents a wall, together with the start cell and the goal cell as (row, column) pairs.

Working: The module works in three steps.

- **Grid generation:** the function create grid builds a list of lists of the requested size and fills it with one value.
- **Start and goal setup:** for random mazes the start is the top-left cell (0, 0) and the goal is the bottom-right cell. For perfect mazes the start is (1, 1) and the goal is (rows-2, cols-2), so that the outer boundary remains a wall.
- **Wall placement:** in a random maze, each cell other than the start and goal becomes a wall with the given probability, and a seed makes the maze reproducible. In a perfect maze, the grid starts as all walls and a randomised depth-first carver removes walls between cells using a stack until every cell has been visited. The user can also click a cell to toggle it between wall and open.

4.2 BFS Search Module

Purpose: To find a path from start to goal by exploring the maze in order of increasing distance from the start.

Inputs: The grid, the start cell and the goal cell.

Outputs: The path from start to goal (empty if none exists) and the list of cells in the order they were expanded.

Working: The module uses a double-ended queue (`collections.deque`) as the frontier.

- **Queue:** the start cell is placed in the queue and in the parent table. At each step the cell at the front is removed, and its unvisited neighbours are appended at the back.
- **Level-by-level exploration:** because new cells join at the back of the queue, all cells at distance k from the start are expanded before any cell at distance $k + 1$. The explored region therefore grows as a wave.
- **Shortest path property:** every cell is first reached by the fewest possible steps, so when the goal is reached the parent chain gives a path with the minimum number of moves. This is guaranteed because all moves have equal cost.

4.3 DFS Search Module

Purpose: To find a path from start to goal by following one route as deep as possible before trying alternatives.

Inputs: The grid, the start cell and the goal cell.

Outputs: The path from start to goal (empty if none exists) and the list of cells in the order they were expanded.

Working: The module uses a Python list as a stack, where each entry stores a cell together with the cell from which it was reached.

- **Stack:** the top entry is popped, and if the cell is already visited it is skipped. Otherwise it is marked visited, its parent is recorded, and its unvisited neighbours are pushed onto the stack.
- **Deep exploration:** since the most recently pushed neighbour is popped next, the search keeps moving forward along the same corridor. The neighbours are pushed in reverse order so that the first direction in the order Up, Right, Down, Left is explored first.
- **Backtracking:** when a cell has no unvisited neighbours, the search reaches a dead end. The next pop returns the most recent unexplored alternative, which is the previous junction, so the search automatically backtracks to it.

4.4 Path Reconstruction Module

Purpose: To convert the parent information recorded during the search into an ordered route from start to goal.

Inputs: The parent table (a dictionary that maps each visited cell to the cell it was reached from), the start cell and the goal cell.

Outputs: A list of cells from start to goal, or an empty list when the goal was never reached.

Working: The module works in two steps.

- **Parent tracking:** during the search every newly discovered cell is stored with its parent, and the start cell has no parent. The table therefore forms a tree rooted at the start.
- **Final path generation:** beginning at the goal, the module follows parent links until it reaches the start, appending each cell to a list. The list is then reversed. The path length is the number of cells minus one, that is, the number of moves.

4.5 Visualization Module

Purpose: To display the maze, the progress of the search and the final path so that the behaviour of each algorithm can be observed.

Inputs: The grid, the start and goal cells, the explored order, the final path, the metrics and the animation delay chosen by the user.

Outputs: A graphical window that shows the coloured maze, the animation of the search, and the statistics of the run.

Working: The module is built with Tkinter.

- **GUI display:** a canvas draws one square per cell, and buttons let the user create a perfect maze, create a random maze, run BFS, run DFS, compare both algorithms and clear the search. A slider controls the animation delay and a label shows the statistics.
- **Colour coding:** white for open cells, near-black for walls, green for the start, red for the goal, light blue for explored cells and yellow for the final path.
- **Search animation:** the stored explored order is replayed with the `after()` method of Tkinter, painting one cell per time step, so the interface stays responsive. When the replay ends, the final path is painted.

4.6 Comparison Module

Purpose: To measure the performance of BFS and DFS on the same maze and present the results side by side.

Inputs: The grid, the start cell, the goal cell and the number of repetitions used for timing.

Outputs: For each algorithm: path length, number of nodes visited and average execution time.

Working: The module calls each algorithm on the same maze and computes three metrics.

- **Path length:** the number of moves in the path returned by the algorithm, which shows how close it is to the shortest route.
- **Nodes visited:** the number of cells expanded before the search stops, which shows how much work was done.
- **Execution time:** the algorithm is repeated several times using `time.perf_counter()` and the average is taken, which reduces the effect of timer resolution and background activity.

Table 4.1 summarises the interface of the modules.

Table 4.1: Summary of module interfaces

Module	Input	Output	Main function
Maze Creation	Rows, columns, density, seed	Grid, start, goal	<code>generate_random_maze</code> , <code>generate_perfect_maze</code>
BFS Search	Grid, start, goal	Path, explored order	<code>bfs</code>
DFS Search	Grid, start, goal	Path, explored order	<code>dfs</code>
Path Reconstruction	Parent table, start, goal	Ordered path	<code>reconstruct_path</code>
Visualization	Grid, explored order, path	Animated display	<code>MazeApp (Tkinter)</code>
Comparison	Grid, start, goal	Length, nodes, time	<code>measure</code>

CHAPTER 5

IMPLEMENTATION AND RESULTS

5.1 EXPERIMENTAL SETUP

The maze solver was implemented as a single Python program that contains the search algorithms, the maze generators, the comparison routine and the graphical interface. Table 5.1 lists the environment used.

Table 5.1: Experimental setup

Item	Description
Programming language	Python 3
GUI toolkit	Tkinter (standard Python library)
Platform	Windows
Algorithms	Breadth-First Search (BFS) and Depth-First Search (DFS)
Libraries used	collections (deque), random, time, tkinter
Metrics	Path length, nodes visited, execution time
Timing method	Average of 300 repeated runs using <code>time.perf_counter()</code>

Maze representation

The maze is stored as a two-dimensional list of integers, where 0 denotes an open cell and 1 denotes a wall. A cell is identified by a (row, column) tuple, which can be used directly as a dictionary key or set member. The parent table is a dictionary that maps a cell to the cell it was reached from, and the visited set and the explored order are kept in a set and a list respectively. This representation needs only $R \times C$ integers and gives constant-time access to any cell.

Neighbour generation

The function `get_neighbors` takes a cell and tries the four direction offsets (-1, 0), (0, 1), (1, 0) and (0, -1), which correspond to Up, Right, Down and Left. A neighbour is returned only if it lies inside the grid and is not a wall. Because

the order of the offsets is fixed, the behaviour of both algorithms is deterministic, and the same maze always gives the same result.

Path reconstruction

During the search each discovered cell is stored with its parent. After the goal is found, the function reconstruct path starts at the goal, follows the parent links back to the start and reverses the list. The number of moves is the length of the list minus one. If the goal never enters the parent table, the function returns an empty list, and the interface reports that the goal is unreachable.

Visualization

The interface draws the grid on a Tkinter canvas using one square of 24 pixels per cell. Before drawing, the search is executed completely and the explored order is stored. The animation then repaints one explored cell at each timer tick using the `after()` method, which keeps the window responsive, and paints the final path in yellow when the replay is complete. The delay between ticks can be changed with a slider, and clicking any cell toggles a wall so that the user can design new test cases.

Pseudocode for BFS

```
BFS(grid, start, goal)
1  queue <- [start]
2  parent[start] <- NIL
3  explored <- empty list
4  while queue is not empty do
5      node <- DEQUEUE(queue)           // front of the queue
6      append node to explored
7      if node = goal then break
8      for each neighbour nb of node in order Up, Right, Down, Left
do
9          if nb is inside grid and nb is open and nb not in parent
then
10             parent[nb] <- node
11             ENQUEUE(queue, nb)       // back of the queue
12  path <- RECONSTRUCT_PATH(parent, start, goal)
13  return path, explored
```

Pseudocode for DFS

```
DFS(grid, start, goal)
1  stack <- [(start, NIL)]
```

```

2  visited <- empty set
3  explored <- empty list
4  while stack is not empty do
5      (node, par) <- POP(stack)           // top of the stack
6      if node in visited then continue
7      add node to visited
8      parent[node] <- par
9      append node to explored
10     if node = goal then break
11     for each neighbour nb of node in reverse order do
12         if nb not in visited then
13             PUSH(stack, (nb, node))
14 path <- RECONSTRUCT_PATH(parent, start, goal)
15 return path, explored

```

Pseudocode for path reconstruction

```

RECONSTRUCT_PATH(parent, start, goal)
1  if goal not in parent then return empty list
2  path <- empty list
3  node <- goal
4  while node is not NIL do
5      append node to path
6      node <- parent[node]
7  reverse path
8  return path

```

The only structural difference between the two algorithms is the frontier. BFS removes cells from the front of a queue while DFS removes them from the top of a stack, and this single change is responsible for all the differences in behaviour that are reported in the next section. Both algorithms visit each cell at most once, so their time complexity is $O(R \times C)$ for a grid of R rows and C columns.

5.2 RESULTS

The program was tested on seven mazes, listed in Table 5.2. They include a hand-crafted maze, two perfect mazes, two mazes with random walls, an empty grid and a maze whose goal is completely enclosed. Every result reported below was produced by running the implementation given in the Appendix. In every test the start cell is the top-left open cell and the goal is the bottom-right open cell. The random seeds used are stated so that the experiments can be repeated.

Table 5.2: Test mazes used in the experiments

Test	Maze type	Size	Seed	Purpose
T1	Hand-crafted 10x10	10 x 10	-	Small hand-made maze
T2	Perfect maze 21x21	21 x 21	7	Single-solution maze
T3	Perfect maze 31x31	31 x 31	21	Larger single-solution maze
T4	Random walls 20x20 (25%)	20 x 20	6	Many alternative routes
T5	Random walls 30x30 (30%)	30 x 30	14	Larger and denser maze
T6	Empty grid 15x15	15 x 15	-	Best case for DFS
T7	Blocked goal 10x10	10 x 10	-	Failure handling

Sample output

The comparison routine prints the metrics in the following form. The same table is shown in a message box when the Compare button is pressed.

```

Test T4: Random walls 20x20 (25%)
Metric      BFS      DFS
Path Length  42      88
Nodes Visited 298     98
Execution Time 0.277 ms 0.118 ms

Test T6: Empty grid 15x15
Metric      BFS      DFS
Path Length  28      28
Nodes Visited 225     29
Execution Time 0.220 ms 0.029 ms

```

Comparison tables

Table 5.3 shows the comparison for test T4, a 20×20 maze with 25% random walls, in the format used by the comparison module.

Table 5.3: BFS versus DFS on test T4 (20 x 20 random maze)

Metric	BFS	DFS
Path Length	42	88
Nodes Visited	298	98
Execution Time	0.277 ms	0.118 ms

Table 5.4 gives the results of all the tests. The path length is the number of moves, and the execution time is the average of 300 runs in milliseconds.

Times depend on the computer used and on background load, so they should be compared with each other rather than treated as absolute values.

Table 5.4: Path length, nodes visited and execution time for all tests

Test	Maze	Path BFS	Path DFS	Nodes BFS	Nodes DFS	Time BFS (ms)	Time DFS (ms)
T1	Hand-crafted 10x10	22	22	53	24	0.053	0.024
T2	Perfect maze 21x21	132	132	182	193	0.164	0.208
T3	Perfect maze 31x31	192	192	305	231	0.281	0.261
T4	Random walls 20x20 (25%)	42	88	298	98	0.277	0.118
T5	Random walls 30x30 (30%)	58	122	592	183	0.554	0.207
T6	Empty grid 15x15	28	28	225	29	0.220	0.029
T7	Blocked goal 10x10	No path	No path	97	97	0.097	0.109

Sample output images

Figure 5.1 shows the result of both algorithms on the random maze T4 and Figure 5.2 shows the result on the perfect maze T2. The remaining output images are given in the Appendix.

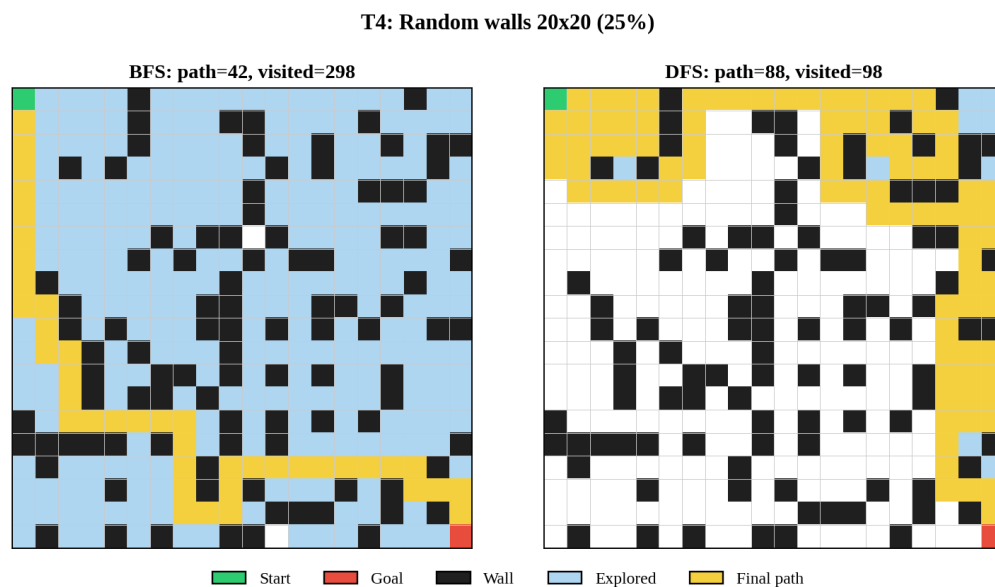


Figure 5.1: BFS and DFS on test T4 (20 x 20 random maze)

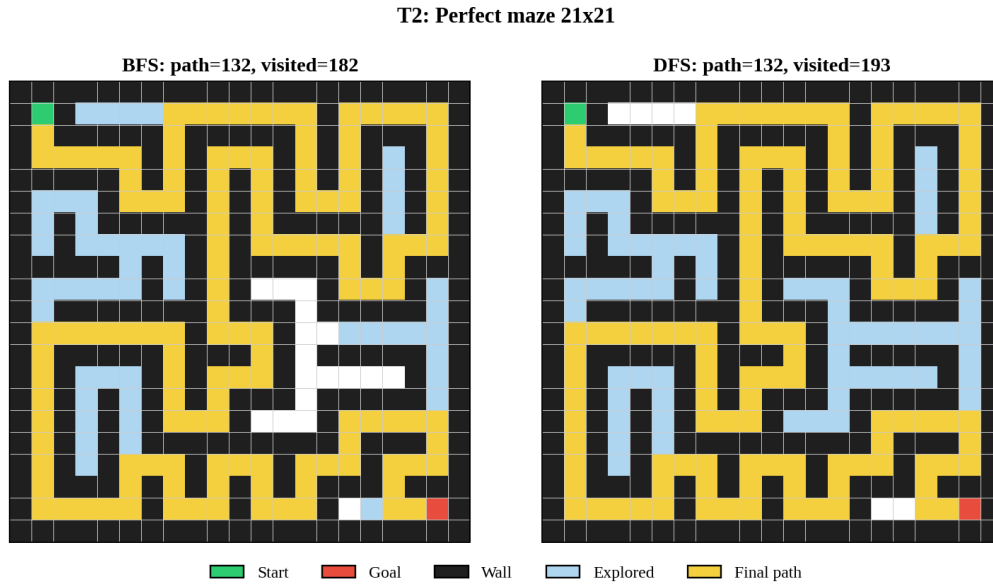


Figure 5.2: BFS and DFS on test T2 (21 x 21 perfect maze)

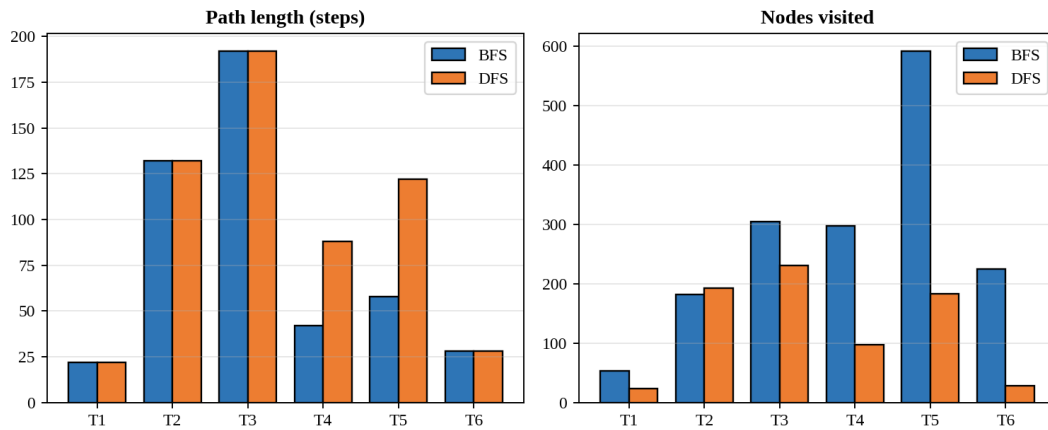


Figure 5.3: Path length and nodes visited for tests T1 to T6

Discussion of observations

- **BFS always found a shortest path.** In the mazes with random walls (T4 and T5), BFS found paths of 42 and 58 moves, whereas DFS found paths of 88 and 122 moves, more than twice as long. DFS accepts the first route it happens to find, which can be a long detour.
- **DFS visited far fewer nodes when many routes exist.** In T4 DFS expanded 98 cells against 298 for BFS, and in T5 it expanded 183 against 592. BFS must examine every cell closer than the goal, which in an open maze is a large fraction of the grid.

- **In the empty grid (T6) the two paths have equal length, but the effort differs greatly.** DFS went almost straight to the goal with 29 cells visited, while BFS expanded 225, that is the entire grid. Here DFS is much cheaper, and the result depends on the fixed neighbour order.
- **In perfect mazes (T2 and T3) both algorithms return the same path.** Only one route exists between any two cells, so the path length is identical (132 and 192 moves). The number of nodes visited is similar for both, and neither algorithm is consistently better, because it depends on how the dead ends are arranged.
- **Execution time follows the number of nodes visited.** The algorithm that expands fewer cells generally finishes sooner, for example DFS in T4, T5 and T6. In T2 and T3 the times are close, and the small differences are within the normal variation of timing.
- **Failure handling works correctly.** In T7, where the goal is enclosed, both algorithms explored all 97 reachable cells and returned an empty path, and the interface reported that no path exists.
- **The trade-off is clear.** BFS gives the best path at the price of exploring more cells and holding a larger frontier, and DFS explores less on average in open mazes but gives no guarantee about the length of its path.

CHAPTER 6

CONCLUSION AND FUTURE WORK

Conclusion

Project summary

This project developed an AI-based maze solver that models a maze as a state space search problem and solves it with Breadth-First Search and Depth-First Search. The system contains a maze creation module for perfect and random mazes, two search modules, a path reconstruction module, a Tkinter visualization module with colour coding and animation, and a comparison module that measures path length, nodes visited and execution time. The tests on seven mazes confirmed that the implementation is correct, including the case in which no path exists.

Learning outcomes

Through the project the team learned how to formulate a real problem as a search problem, how the choice of data structure for the frontier determines the behaviour of an algorithm, how parent pointers are used to recover a solution, how to measure algorithms in a fair way and how to present the behaviour of an algorithm through a graphical interface. The project also gave practical experience with Python data structures, modular design and GUI programming.

BFS advantages

- It is complete and always finds a shortest path when all moves have equal cost.
- Its behaviour is predictable, because it expands cells in order of distance from the start.

- It is the basis of wavefront planning, flood filling and shortest-path algorithms such as Dijkstra.

DFS advantages

- It needs much less memory, because the frontier contains only the current branch and its pending alternatives.
- It can reach the goal after examining very few cells when the layout suits it, as in the empty grid and the random mazes tested here.
- It is simple to implement, by recursion or with an explicit stack, and it is the basis of backtracking, cycle detection and maze generation.

AI relevance

The project shows the core idea of problem-solving agents: describe the states, actions and goal, and then let a search procedure find the sequence of actions. The comparison of BFS and DFS demonstrates the properties by which AI search algorithms are judged, namely completeness, optimality, time and space. The project therefore gives a firm foundation for the informed search methods and planning techniques studied later in Artificial Intelligence.

Future Work

- **A* Algorithm:** adding a heuristic such as the Manhattan distance would let the solver find shortest paths while examining far fewer cells than BFS.
- **Dijkstra Algorithm:** supporting cells with different movement costs, such as mud or water, would extend the system to weighted grids.
- **Dynamic Maze Generation:** adding more generators and allowing walls to appear and disappear during the search would test how the algorithms cope with changing environments.

- **Real-time Robot Navigation:** the planner could be connected to a small mobile robot that uses sensors to build the grid and then follows the computed path.
- **3D Maze Solver:** extending the grid to several levels with six directions of movement would model multi-floor buildings and three-dimensional spaces.

CHAPTER 7

REFERENCES

Buck, J. (2015) *Mazes for Programmers: Code Your Own Twisty Little Passages*. Raleigh, NC: The Pragmatic Bookshelf.

Choset, H., Lynch, K.M., Hutchinson, S., Kantor, G., Burgard, W., Kavraki, L.E. and Thrun, S. (2005) *Principles of Robot Motion: Theory, Algorithms, and Implementations*. Cambridge, MA: MIT Press.

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2009) *Introduction to Algorithms*, 3rd edn. Cambridge, MA: MIT Press.

Dijkstra, E.W. (1959) 'A note on two problems in connexion with graphs', *Numerische Mathematik*, Vol.1, No.1, pp.269-271.

Edmonds, J. and Karp, R.M. (1972) 'Theoretical improvements in algorithmic efficiency for network flow problems', *Journal of the ACM*, Vol.19, No.2, pp.248-264.

Hart, P.E., Nilsson, N.J. and Raphael, B. (1968) 'A formal basis for the heuristic determination of minimum cost paths', *IEEE Transactions on Systems Science and Cybernetics*, Vol.4, No.2, pp.100-107.

Hopcroft, J. and Tarjan, R. (1973) 'Algorithm 447: Efficient algorithms for graph manipulation', *Communications of the ACM*, Vol.16, No.6, pp.372-378.

Korf, R.E. (1985) 'Depth-first iterative-deepening: An optimal admissible tree search', *Artificial Intelligence*, Vol.27, No.1, pp.97-109.

Lee, C.Y. (1961) 'An algorithm for path connections and its applications', *IRE Transactions on Electronic Computers*, Vol.EC-10, No.3, pp.346-365.

Moore, E.F. (1959) 'The shortest path through a maze', *Proceedings of an International Symposium on the Theory of Switching, Part II*. Cambridge, MA: Harvard University Press, pp.285-292.

Nilsson, N.J. (1998) *Artificial Intelligence: A New Synthesis*. San Francisco, CA: Morgan Kaufmann.

- Pearl, J. (1984) *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Reading, MA: Addison-Wesley.
- Poole, D.L. and Mackworth, A.K. (2017) *Artificial Intelligence: Foundations of Computational Agents*, 2nd edn. Cambridge: Cambridge University Press.
- Python Software Foundation (2024) *collections* — *Container datatypes*, Python 3 Documentation. Available at:
<https://docs.python.org/3/library/collections.html>
- Python Software Foundation (2024) *tkinter* — *Python interface to Tcl/Tk*, Python 3 Documentation. Available at:
<https://docs.python.org/3/library/tkinter.html>
- Russell, S. and Norvig, P. (2021) *Artificial Intelligence: A Modern Approach*, 4th edn. Harlow: Pearson.
- Skiena, S.S. (2008) *The Algorithm Design Manual*, 2nd edn. London: Springer.
- Sturtevant, N.R. (2012) ‘Benchmarks for grid-based pathfinding’, *IEEE Transactions on Computational Intelligence and AI in Games*, Vol.4, No.2, pp.144-148.
- Tarjan, R. (1972) ‘Depth-first search and linear graph algorithms’, *SIAM Journal on Computing*, Vol.1, No.2, pp.146-160.

APPENDIX

A. Source Code

The complete source code of the maze solver (file maze_solver.py) is given below. It runs on Python 3 under Windows with the standard library only. Running the file opens the Tkinter interface, and the functions bfs, dfs and measure can also be imported and used without a display.

```
"""
AI-Based Maze Solver Using Breadth-First Search (BFS) and Depth-First Search (DFS)
Foundations of Artificial Intelligence - Rajalakshmi Engineering College
Authors : Kamalesh V (240701224), Karthick R (240701232)
Guide   : Muneeshwari P
"""

import random
import time
from collections import deque

OPEN, WALL = 0, 1
# Neighbour order: Up, Right, Down, Left
DIRECTIONS = [(-1, 0), (0, 1), (1, 0), (0, -1)]

# -----
# 4.1 MAZE CREATION MODULE
# -----
def create_grid(rows, cols, fill=OPEN):
    """Return a rows x cols grid filled with the given value."""
    return [[fill for _ in range(cols)] for _ in range(rows)]

def generate_random_maze(rows, cols, density=0.25, seed=None):
    """Random wall placement. Start = (0,0), Goal = (rows-1, cols-1)."""
    rng = random.Random(seed)
    grid = create_grid(rows, cols)
    start, goal = (0, 0), (rows - 1, cols - 1)
    for r in range(rows):
        for c in range(cols):
            if (r, c) not in (start, goal) and rng.random() < density:
                grid[r][c] = WALL
    return grid, start, goal

def generate_perfect_maze(rows, cols, seed=None):
    """Perfect maze (exactly one path between any two cells) using an
    iterative randomised depth-first carver. rows and cols must be odd."""
    rng = random.Random(seed)
    grid = create_grid(rows, cols, WALL)
    grid[1][1] = OPEN
    stack = [(1, 1)]
    while stack:
        r, c = stack[-1]
        options = []
        for dr, dc in DIRECTIONS:
            nr, nc = r + 2 * dr, c + 2 * dc
            if 0 < nr < rows - 1 and 0 < nc < cols - 1 and grid[nr][nc] == WALL:
                options.append((dr, dc))
        if options:
```

```

        dr, dc = rng.choice(options)
        grid[r + dr][c + dc] = OPEN
        grid[r + 2 * dr][c + 2 * dc] = OPEN
        stack.append((r + 2 * dr, c + 2 * dc))
    else:
        stack.pop()
    return grid, (1, 1), (rows - 2, cols - 2)

def get_neighbors(grid, cell):
    """Return the walkable 4-connected neighbours of a cell."""
    r, c = cell
    result = []
    for dr, dc in DIRECTIONS:
        nr, nc = r + dr, c + dc
        inside = 0 <= nr < len(grid) and 0 <= nc < len(grid[0])
        if inside and grid[nr][nc] == OPEN:
            result.append((nr, nc))
    return result

# -----
# 4.4 PATH RECONSTRUCTION MODULE
# -----
def reconstruct_path(parent, start, goal):
    """Walk back from goal to start through the parent map."""
    if goal not in parent:
        return []
    path, node = [], goal
    while node is not None:
        path.append(node)
        node = parent[node]
    path.reverse()
    return path if path[0] == start else []

# -----
# 4.2 BFS SEARCH MODULE (FIFO queue)
# -----
def bfs(grid, start, goal):
    """Returns (path, explored_order)."""
    queue = deque([start])
    parent = {start: None}
    explored = []
    while queue:
        node = queue.popleft()
        explored.append(node)
        if node == goal:
            break
        for nb in get_neighbors(grid, node):
            if nb not in parent:
                parent[nb] = node
                queue.append(nb)
    return reconstruct_path(parent, start, goal), explored

# -----
# 4.3 DFS SEARCH MODULE (LIFO stack)
# -----
def dfs(grid, start, goal):
    """Returns (path, explored_order)."""
    stack = [(start, None)]
    parent, explored, visited = {}, [], set()
    while stack:
        node, par = stack.pop()
        if node in visited:
            continue
        visited.add(node)
        parent[node] = par
        explored.append(node)

```

```

        if node == goal:
            break
        # reversed() so that the first direction (Up) is explored first
        for nb in reversed(get_neighbors(grid, node)):
            if nb not in visited:
                stack.append((nb, node))
    return reconstruct_path(parent, start, goal), explored

# -----
# 4.6 COMPARISON MODULE
# -----
def measure(algorithm, grid, start, goal, repeats=200):
    """Run an algorithm and return path length, nodes visited, avg time (s)."""
    t0 = time.perf_counter()
    for _ in range(repeats):
        path, explored = algorithm(grid, start, goal)
    elapsed = (time.perf_counter() - t0) / repeats
    return {"path_length": max(len(path) - 1, 0) if path else None,
            "nodes visited": len(explored),
            "time": elapsed}

# -----
# 4.5 VISUALIZATION MODULE (Tkinter GUI)
# -----
def run_gui():
    import tkinter as tk
    from tkinter import messagebox

    COLORS = {"open": "#FFFFFF", "wall": "#1F1F1F", "start": "#2ECC71",
              "goal": "#E74C3C", "explored": "#AED6F1", "path": "#F4D03F"}
    CELL = 24

    class MazeApp:
        def __init__(self, root):
            self.root = root
            root.title("AI-Based Maze Solver - BFS and DFS")
            self.rows, self.cols = 21, 21
            self.delay = 15
            self.job = None
            self.canvas = tk.Canvas(root, width=self.cols * CELL,
                                    height=self.rows * CELL, bg="white")
            self.canvas.grid(row=0, column=0, rowspan=12, padx=8, pady=8)
            self.canvas.bind("<Button-1>", self.toggle_wall)
            buttons = [("Perfect Maze", self.new_perfect),
                      ("Random Maze", self.new_random),
                      ("Solve with BFS", lambda: self.solve(bfs, "BFS")),
                      ("Solve with DFS", lambda: self.solve(DFS, "DFS")),
                      ("Compare BFS vs DFS", self.compare),
                      ("Clear Search", self.draw)]
            for i, (txt, cmd) in enumerate(buttons):
                btn = tk.Button(root, text=txt, width=20, command=cmd)
                btn.grid(row=i, column=1, padx=6, pady=3)
            self.speed = tk.Scale(root, from_=1, to=100, orient="horizontal",
                                  label="Delay (ms)", command=self.set_delay)
            self.speed.set(self.delay)
            self.speed.grid(row=6, column=1)
            self.info = tk.Label(root, text="Click cells to toggle walls",
                                  justify="left", font=("Times New Roman", 11))
            self.info.grid(row=7, column=1, rowspan=4)
            self.new_perfect()

        def set_delay(self, v):
            self.delay = int(v)

        def new_perfect(self):
            self.grid, self.start, self.goal = generate_perfect_maze(
                self.rows, self.cols)
            self.draw()

```

```

def new_random(self):
    self.grid, self.start, self.goal = generate_random_maze(
        self.rows, self.cols, 0.28)
    self.draw()

def toggle_wall(self, event):
    r, c = event.y // CELL, event.x // CELL
    inside = 0 <= r < self.rows and 0 <= c < self.cols
    if inside and (r, c) not in (self.start, self.goal):
        self.grid[r][c] ^= 1
        self.draw()

def paint(self, cell, colour):
    r, c = cell
    self.canvas.create_rectangle(c * CELL, r * CELL,
                                (c + 1) * CELL, (r + 1) * CELL,
                                fill=colour, outline="#CCCCCC")

def draw(self):
    if self.job:
        self.root.after_cancel(self.job)
        self.job = None
    self.canvas.delete("all")
    for r in range(self.rows):
        for c in range(self.cols):
            kind = "wall" if self.grid[r][c] else "open"
            self.paint((r, c), COLORS[kind])
    self.paint(self.start, COLORS["start"])
    self.paint(self.goal, COLORS["goal"])

def solve(self, algo, name):
    self.draw()
    path, explored = algo(self.grid, self.start, self.goal)
    stats = measure(algo, self.grid, self.start, self.goal)
    if not path:
        messagebox.showinfo("No Path", "Goal is unreachable.")
        return
    self.info.config(text=(
        f"{name}\nPath length : {stats['path_length']}\n"
        f"Nodes visited: {stats['nodes visited']}\n"
        f"Time : {stats['time'] * 1000:.3f} ms"))
    self.animate(explored, path, 0)

def animate(self, explored, path, i):
    if i < len(explored):
        # search animation
        if explored[i] not in (self.start, self.goal):
            self.paint(explored[i], COLORS["explored"])
            self.job = self.root.after(self.delay, self.animate,
                                       explored, path, i + 1)
    else:
        # final path
        for cell in path:
            if cell not in (self.start, self.goal):
                self.paint(cell, COLORS["path"])

def compare(self):
    b = measure(bfs, self.grid, self.start, self.goal)
    d = measure(dfs, self.grid, self.start, self.goal)
    messagebox.showinfo("Comparison", (
        "Metric          BFS          DFS\n"
        f"Path Length       {b['path length']}       {d['path length']}\n"
        f"Nodes Visited      {b['nodes visited']}       {d['nodes visited']}\n"
        f"Time (ms)          {b['time']*1000:.3f}       {d['time']*1000:.3f}"))

root = tk.Tk()
MazeApp(root)
root.mainloop()

```

```
if name == "main":
    run_gui()
```

B. Screenshots

Figure B.1 shows a snapshot of the search animation on test T4 after 40% of the exploration, using the colour scheme of the interface. BFS has grown a compact wave around the start cell, while DFS has already pushed a long branch far across the maze.

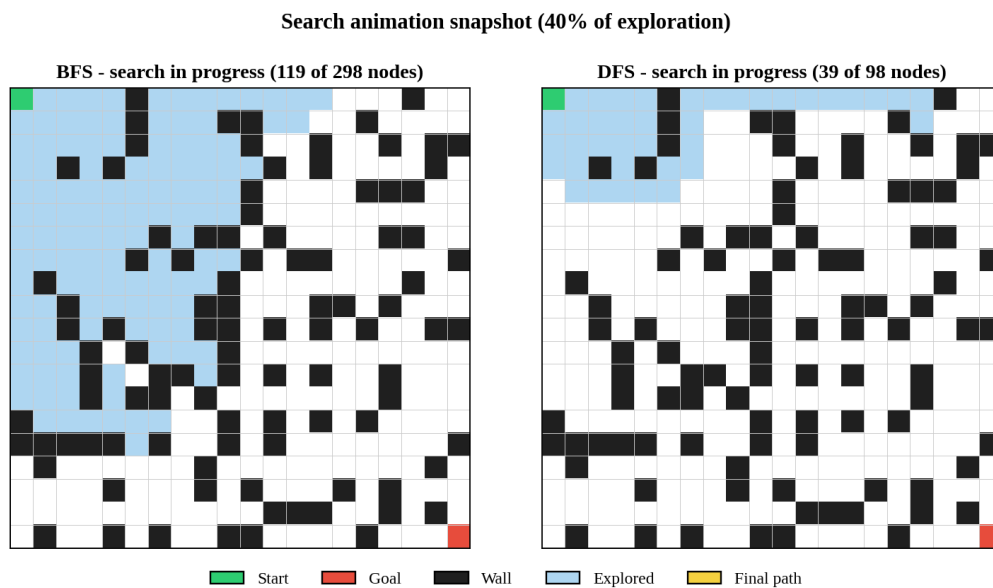


Figure B.1: Search animation snapshot for BFS and DFS on test T4

C. Test Cases

Table C.1 lists the test cases, the expected behaviour and the observed result. All tests behaved as expected.

Table C.1: Test cases and observed results

ID	Test case	Expected behaviour	Observed result
T1	Hand-crafted 10 x 10 maze	Both algorithms find a valid path	Both found a path of 22 moves
T2	Perfect maze 21 x 21, seed 7	Both find the unique path	Both found 132 moves
T3	Perfect maze 31 x 31, seed 21	Both find the unique path	Both found 192 moves
T4	Random walls 20 x 20, 25%	BFS path is not longer than DFS path	BFS 42, DFS 88 moves
T5	Random walls 30 x 30, 30%	BFS path is not longer than DFS path	BFS 58, DFS 122 moves
T6	Empty grid 15 x 15	Path of 28 moves (Manhattan distance)	Both found 28 moves
T7	Goal enclosed by walls	Empty path and a "No Path" message	Both returned an empty path

D. Output Images

Figures D.1 to D.7 show the output of BFS and DFS on all the test mazes. Explored cells are shown in light blue and the final path in yellow.

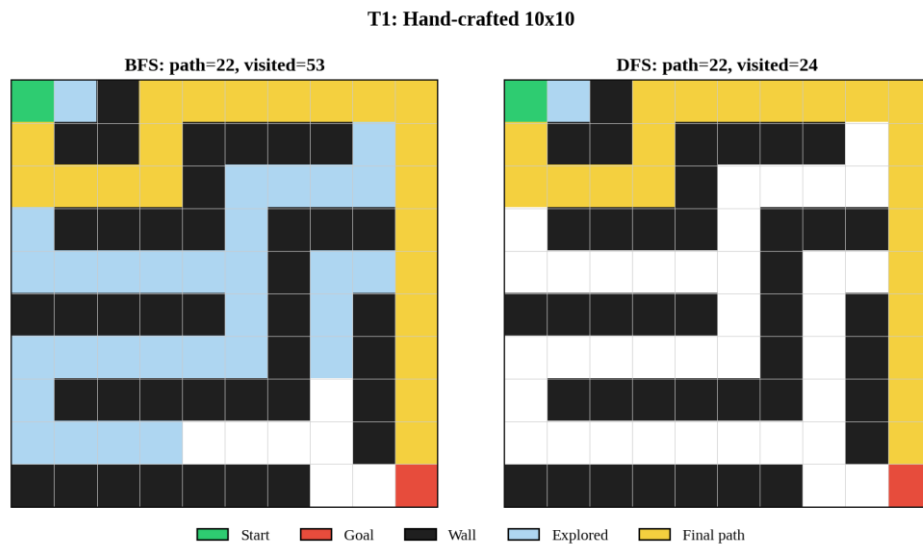


Figure D.1: Output for test T1 (Hand-crafted 10x10)

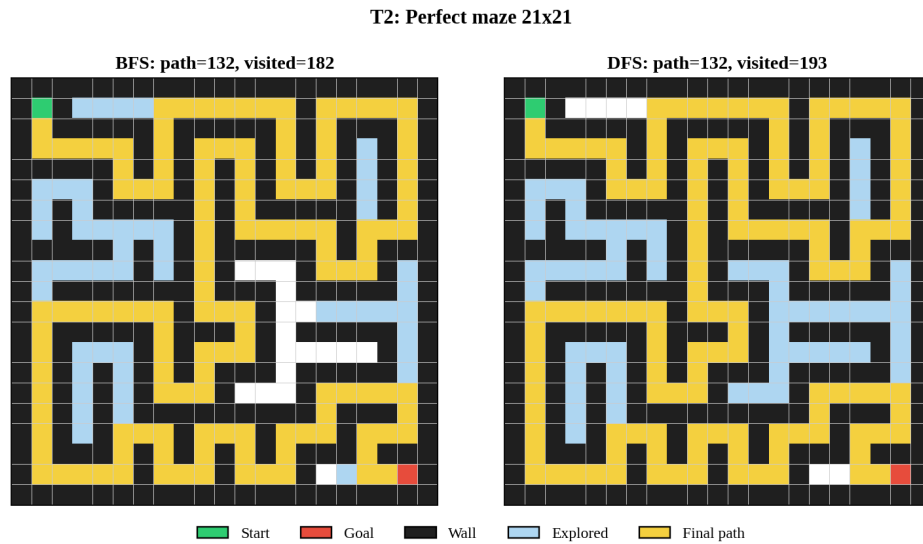


Figure D.2: Output for test T2 (Perfect maze 21x21)

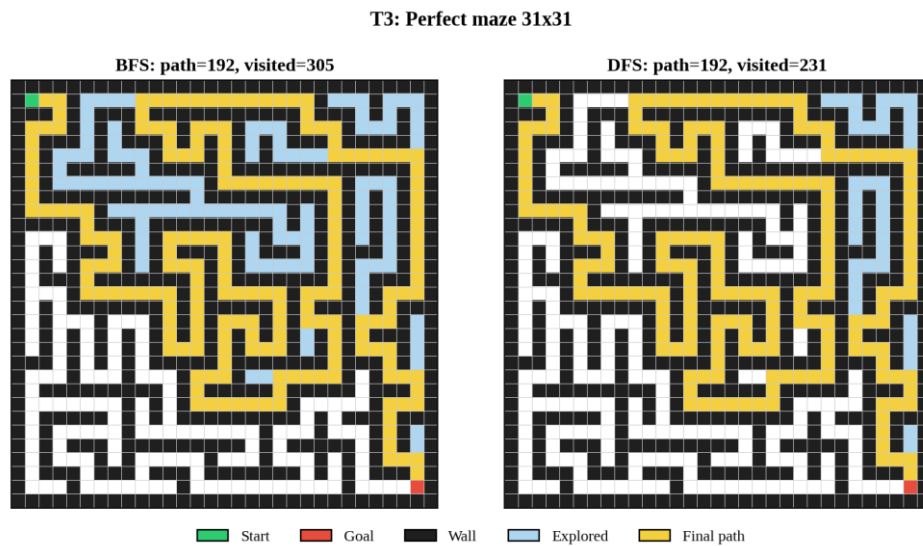


Figure D.3: Output for test T3 (Perfect maze 31x31)

T4: Random walls 20x20 (25%)

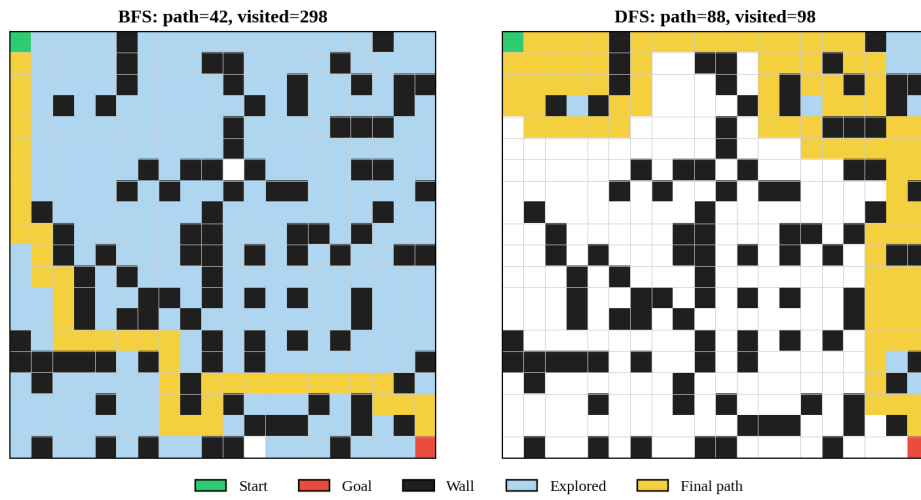


Figure D.4: Output for test T4 (Random walls 20x20 (25%))

T5: Random walls 30x30 (30%)

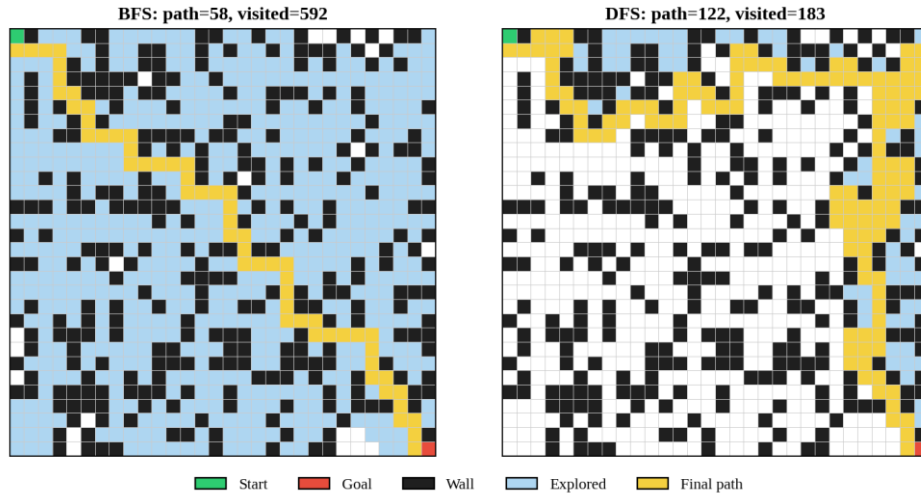


Figure D.5: Output for test T5 (Random walls 30x30 (30%))

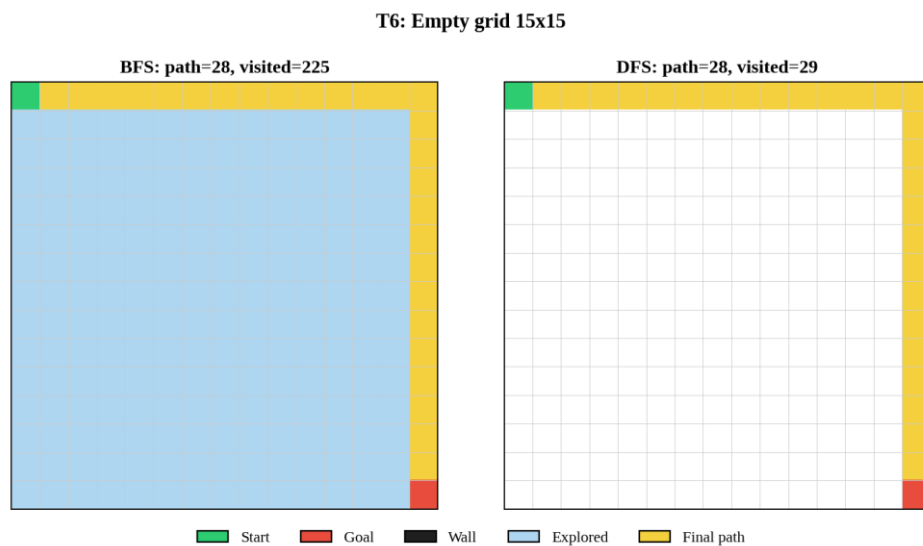


Figure D.6: Output for test T6 (Empty grid 15x15)

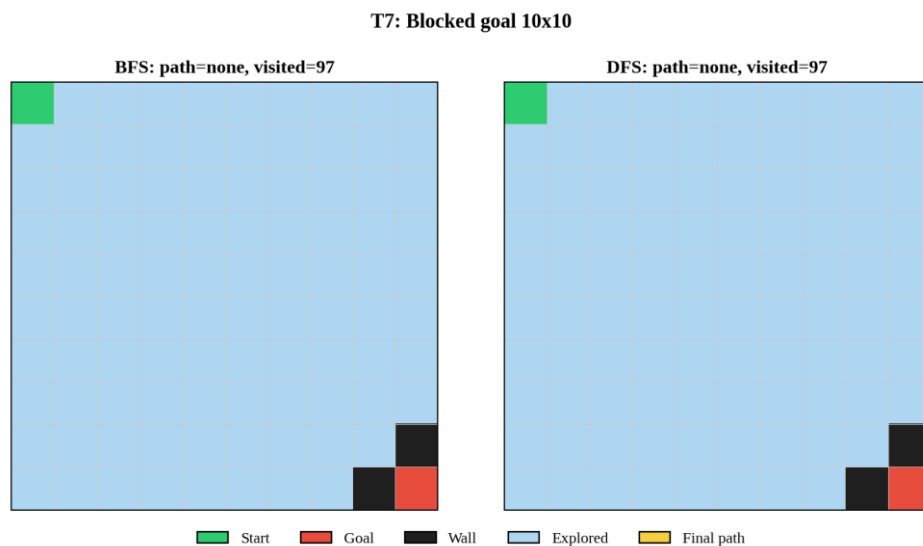


Figure D.7: Output for test T7 (Blocked goal 10x10)